

# claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

## Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_f39953fd-98d\agent-ac2611e6ea7d84e1e.jsonl`
- SHA-256: `e9253b7e65489523edccff1c6f72bfa09366a722a6d457a785353589fe156203`
- Source modified: `2026-06-22T15:05:03+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_f39953fd-98d`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

## Transcript

### 1. user (2026-06-22T15:00:24.575Z)

Review the cloud-dispatch wiring for CORRECTNESS + EDGE CASES. Run ``cd C:/Users/avidu/Projects/bhi-cloudrun && git diff origin/master...HEAD`` and read backend/main.py (`_maybe_dispatch_remote`, `_ingest_remote_segments`, the gate inside `_execute_processing`) + the cloud contract (backend/compute/cloud\_run.py `run_infer`, `base.py ComputeResult`) in `C:/Users/avidu/Projects/bhi-cloudrun`. Scrutinize: (1) `video_id` consistency ? the API ingests under its LOCAL `video_id` but reads `result.outputs_uri` which uses `result.video_id`; what if they differ (the cloud job hashes different bytes)? (2) the destroy-after-confirm lifecycle on the cloud path ? is `play_wm` captured before any cloud write, does `_finalize_reingest` drop the right rows, does the failure path prune correctly + preserve prior good rows? (3) the segments list built via `query_play_segments` filtered by `id>play_wm` ? correct + ordered? (4) `segmentation_ran/segmenter_actual` for the report; (5) the `returncode!=0` + exception paths. Report concrete findings (file:line). Empty list if none.

### 2. assistant (2026-06-22T15:00:27.101Z)

I'll review the cloud-dispatch wiring. Let me start by examining the diff and the relevant files.

### 3. user (2026-06-22T15:00:30.663Z)

```
---BRANCH---
feat/serving-api-cloud-dispatch
---HEAD---
304252b feat(serving): wire /api/process to dispatch DETECT to Cloud Run (M6 -> API), default-OFF
---DIFFSTAT---
 backend/main.py | 100 +++++
 tests/test_api_cloud_dispatch.py | 212 +++++
 2 files changed, 312 insertions(+)
```

### 4. assistant (2026-06-22T15:00:34.350Z)

Let me read the diff in detail and the relevant source files.

### 5. user (2026-06-22T15:00:35.619Z)

```
diff --git a/backend/main.py b/backend/main.py
index 4e5a19a..04ffb67 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -1,10 +1,12 @@
 import os
 import sys
```

```

import argparse
+import csv
import json
import logging
import shutil
import socket
+import tempfile
import threading
import uuid
import webbrowser
@@ -436,6 +438,94 @@ def download_reel(filename: str):
    return FileResponse(path, media_type=media, filename=name)

+def _ingest_remote_segments(video_id: str, outputs_uri: str, storage_config: dict) -> int:
+    """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from the
+    bucket and
+    persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters call ? so
+    ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they were
+    produced.
+    Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst pattern
+    (cloud
+    backends REQUIRE a dst). A malformed row is skipped, never aborts the ingest."""
+    uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
+    ref = storage.parse_uri(uri)
+    dst = os.path.join(tempfile.mkdtemp(prefix="cr-ingest-"), ref.name or "intervals.csv")
+    local = storage.fetch(uri, dst, config=storage_config)
+    n = 0
+    with open(local, newline="", encoding="utf-8") as f:
+        for row in csv.DictReader(f):
+            try:
+                start, end = float(row["start"]), float(row["end"])
+            except (KeyError, TypeError, ValueError):
+                continue
+            meta: Dict[str, Any] = {"source": "cloud_run",
+                                   "ending_reason": (row.get("ending_reason") or "").strip()}
+            sc = (row.get("shot_count") or "").strip()
+            if sc:
+                try:
+                    meta["shot_count"] = int(float(sc))
+                except ValueError:
+                    meta["shot_count"] = sc
+            db_instance.add_play_segment(video_id, start, end, metadata=meta)
+            n += 1
+    return n
+
+def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name: str, val_results,
+                             play_wm: int, cand_wm: int, notify) -> Optional[Dict[str, Any]]:
+    """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
+    ``deployment.compute_target
+    == "cloud_run``, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
+    intervals into
+    THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
+
+    Returns a response dict (``success`` | ``failed``) when the cloud path is taken, or
+    ``None`` to
+    fall through to the in-process segmenter ? **DEFAULT-OFF**: ``get_compute_backend`` returns
+    ``None``
+    for every non-``cloud_run`` target, so the in-process path stays byte-identical."""
+    from backend.compute import ComputeJobSpec, get_compute_backend
+
+    compute = get_compute_backend(global_config)
+    if compute is None:
+        return None # default-OFF ? run the in-process segmenter below (unchanged)

```

```

+
+ storage_cfg = getattr(global_config, "storage", None)
+
+ def _failed(msg: str) -> Dict[str, Any]:
+     # Shape-match the in-process segmenter-fail branch + restore prior good rows (none
written yet).
+     db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
+     return {"video_id": video_id, "status": "failed", "message": msg,
+           "results": [r.model_dump() for r in val_results], "play_segments": []}
+
+ # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local to this
box.
+ try:
+     input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
+ except ValueError as e:
+     return _failed(f"cloud-serving: unresolvable input '{req.video_path}': {e}")
+ if input_ref.backend == "local":
+     return _failed("cloud-serving needs a cloud input (e.g. gs://?); a path local to this
machine "
+                   "can't be read by the Cloud Run job. Put the video in your bucket
first.")
+ out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
+ if not out_root:
+     return _failed("cloud-serving needs storage.output_root set to a bucket (e.g.
gs://your-bucket) "
+                   "? that's where the Cloud Run job writes the result.")
+
+ notify("dispatching (cloud_run)")
+ logger.info("[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
req.video_path, out_root)
+ spec = ComputeJobSpec(video_uri=req.video_path, out_uri=out_root, segmenter=seg_name)
+ try:
+     result = compute.run_infer(spec) # dispatch the Cloud Run Job + bucket-poll to
completion
+ except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job failure,
not a 500
+     logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
+     return _failed(f"cloud dispatch error: {e}")
+ if result.returncode != 0:
+     return _failed(f"cloud job failed (returncode {result.returncode}).")
+
+ notify("ingesting (cloud_run)")
+ storage_dict = storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else {}
+ _ingest_remote_segments(video_id, result.outputs_uri, storage_dict)
+ # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then drops the
old ones.
+ segments = [s for s in db_instance.query_play_segments(video_id, {}) if int(s.get("id", 0))
> play_wm]
+ _finalize_reingest(video_id, segments, play_wm, cand_wm)
+ return {"video_id": video_id, "status": "success",
+       "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
+       "results": [r.model_dump() for r in val_results], "play_segments": segments}
+
+
+ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
+     """The full hash ? validate ? segment ? report pipeline for one video.
@@ -508,6 +598,16 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
+     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old only
+     # if the run yields segments; otherwise drop the new partial rows and keep the old.
play_wm, cand_wm = db_instance.segment_watermarks(video_id)
+     # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run, run the
heavy

```

```

+         # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
+         # DEFAULT-OFF: returns None for every non-cloud target ? the in-process segmenter below
runs
+         # byte-identically. (player_pool/max_frames aren't threaded to the worker `infer` CLI
yet, so
+         # they're dropped on the cloud path in this slice ? a documented follow-up.)
+         remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm, cand_wm,
notify)
+         if remote is not None:
+             segmentation_ran = True
+             response = remote
+             return response
        segmenter = segmenter_cls(db_instance, global_config)
        try:
            result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)

```

## 6. assistant (2026-06-22T15:00:37.770Z)

Now let me read the full `\_execute\_processing` function for context, plus the helper functions referenced.

## 7. user (2026-06-22T15:00:38.758Z)

```

525         "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
526         "results": [r.model_dump() for r in val_results], "play_segments": segments}
527
528
529     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
530         """The full hash ? validate ? segment ? report pipeline for one video.
531
532         This is the former synchronous /api/process body, unchanged in behavior, now run on
533         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
534         the sync endpoint used to return (UI compatibility); the per-video observability
535         report is still emitted in `finally:` and grafted as response["reports"].
536
537         `progress` is an optional callable(stage: str) used to surface coarse job progress.
538         Raises on unexpected internal errors ? the caller records those as a job failure
539         (after this function has already restored the pre-run segments, see below).
540         """
541         notify = progress or (lambda stage: None)
542
543         notify("hashing")
544         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
545         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
546         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
547         # consumers (hash / validators / segmenter) use the resolved local path.
548         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
549         video_id = compute_video_id(local_video)
550         was_reingest = db_instance.get_video(video_id) is not None
551
552         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
553         notify("validating")
554         logger.info(f"Running validations for {req.video_path}...")
555         val_results, val_context = registry.run_all_with_context(local_video, global_config)
556         failures = [r for r in val_results if not r.passed]
557
558         # typed AppConfig: attribute access for the default segmenter (with safe fallback)

```

```

559     default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
560     seg_name = req.segmenter or default_seg
561     segmenter_actual = None
562     segmentation_ran = False
563     response: Optional[Dict[str, Any]] = None
564     try:
565         if failures:
566             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
567             # status; it no longer destroys the video's prior good segments.
568             db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
569             response = {
570                 "video_id": video_id,
571                 "status": "failed",
572                 "message": "Video failed validation checks.",
573                 "results": [r.model_dump() for r in val_results],
574             }
575             return response
576
577         # 2. Run segmenter & indexer
578         logger.info("[API] Video passed checks. Segmenting and indexing...")
579         db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
580
581         segmenter_cls = segmenter_registry.get(seg_name)
582         if not segmenter_cls:
583             # Submit-time validation already 400s unknown names; this is the defensive
584             # in-worker path (e.g. config default pointing at an unregistered
segmenter).
585             available = list(segmenter_registry.list_available().keys())
586             response = {
587                 "video_id": video_id,
588                 "status": "failed",
589                 "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
590                 "results": [r.model_dump() for r in val_results],
591                 "play_segments": [],
592             }
593             return response
594
595         logger.info(f"[API] Using segmenter: {seg_name}")
596         notify(f"segmenting ({seg_name})")
597         segmenter_actual = seg_name
598         # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
599         # if the run yields segments; otherwise drop the new partial rows and keep the
old.
600         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
601         # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
602         # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
603         # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
604         # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
605         # they're dropped on the cloud path in this slice ? a documented follow-up.)
606         remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm,
cand_wm, notify)
607         if remote is not None:
608             segmentation_ran = True
609             response = remote
610             return response
611         segmenter = segmenter_cls(db_instance, global_config)

```

```

612         try:
613             result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
614         except Exception:
615             # A raising segmenter must not leave half-written rows: restore the pre-run
616             # state (same destroy-after-confirm contract as the Failure branch), then
617             let
618                 # the job worker record the failure.
619                 db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
620                 raise
621             segmentation_ran = True
622
623         if isinstance(result, Failure):
624             logger.error(f"[API] Segmenter failed: {result.message}")
625             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
626             response = {
627                 "video_id": video_id,
628                 "status": "failed",
629                 "message": f"Segmenter '{seg_name}' failed: {result.message}",
630                 "results": [r.model_dump() for r in val_results],
631                 "play_segments": [],
632             }
633             return response
634
635         segments = result.value if hasattr(result, "value") else result
636         notify("finalizing")
637         _finalize_reingest(video_id, segments, play_wm, cand_wm)
638
639         response = {
640             "video_id": video_id,
641             "status": "success",
642             "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
643             "results": [r.model_dump() for r in val_results],
644             "play_segments": segments,
645         }
646         return response
647     finally:
648         # Always emit the per-video observability report (next to the video, or to
649         # report.output_dir). Never raises. Surface the paths in the response.
650         paths = emit_indexer_report(
651             db=db_instance, video_id=video_id, video_path=req.video_path,
652             config=global_config,
653             val_results=val_results, val_context=val_context,
654             segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
655             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
656         )
657         if paths and isinstance(response, dict):
658             response["reports"] = paths
659
660 @app.post("/api/process", status_code=202)
661 def process_video(req: ProcessRequest, request: Request):
662     """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
663
664     Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
665     client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
666     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
667     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
668     """
669     # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
670     single-user,
671     # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
672     missing or
673     # spoofed identity header fails here with 401, before any work) and tags the job's

```

```

owner_id.
671         try:
672             owner_id = edge_auth.resolve_tenant(request.headers, global_config)
673         except edge_auth.EdgeAuthError as e:
674             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
675
676         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
677         try:
678             validate_input_path(req.video_path, allowed_root=allowed_root)
679             # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
680             # scheme, which we map to a clean 400 (an unknown scheme must never 500).
681             input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
682         except ValueError as e:
683             raise HTTPException(status_code=400, detail=str(e)) from e
684         # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path

```

## 8. assistant (2026-06-22T15:00:41.409Z)

Now let me look at the cloud contract files: `cloud\_run.py` (run\_infer) and `base.py` (ComputeResult/ComputeJobSpec).

## 9. user (2026-06-22T15:00:41.858Z)

```

1         """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3         COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4         (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure
DETECT?WINDOW?STITCH
5         core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
6         (cloud-serving) ? without the core ever branching on the profile (D1).
7
8         The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
9         non-local backend. Every other target (``"" / ``local_gpu`` / ``cpu_mock``) keeps the
worker's
10        existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
inline").
11        """
12
13        from __future__ import annotations
14
15        from abc import ABC, abstractmethod
16        from dataclasses import dataclass, field
17        from typing import Optional
18
19
20        @dataclass(frozen=True)
21        class ComputeJobSpec:
22            """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
23            from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
24            deterministic regardless of where the job ran."""
25
26            video_uri: str # input (gs://? for cloud; a path/local://
for local)
27            out_uri: str # output ROOT (gs://? bucket for cloud)
28            segmenter: Optional[str] = None
29            # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
30            config_overrides: tuple[str, ...] = ()
31

```

```

32
33     @dataclass(frozen=True)
34     class ComputeResult:
35         """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
36         (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch
timeout or an
37         empty/unreadable completion marker** (the cloud path only). ``video_id`` is known
once the job's
38         done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
39
40         returncode: int
41         outputs_uri: str = ""
42         video_id: str = ""
43         report: dict = field(default_factory=dict)
44
45
46     class ComputeBackend(ABC):
47         """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
48         its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
49         execution identically."""
50
51         @abstractmethod
52         def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
53             """Run ``spec`` to completion and return its ``ComputeResult``."""
54

```

#### 10. assistant (2026-06-22T15:00:44.806Z)

Key insight confirmed: `ComputeJobSpec.video\_id` docstring says the worker re-computes the MD5 from the same input bytes, and output lands at `<out\_uri>/outputs/<video\_id>/`. Now let me read `cloud\_run.py`'s `run\_infer`.

#### 11. user (2026-06-22T15:00:45.283Z)

```

1     """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2     for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4     Flow (the cloud-serving compute path):
5     1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6         --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7     2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8         completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9         it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10        cold-start is absorbed by the async poll.
11    3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
12        ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
13
14    The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
15    so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
16
17    The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
18    with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-

```



```

run`` and is
19     owner-verified on the M7 real run.
20     ""
21
22     from __future__ import annotations
23
24     import json
25     import logging
26     import os
27     import tempfile
28     import uuid
29     from abc import ABC, abstractmethod
30     from typing import Any, List, Optional
31
32     from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
33     from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy,
poll_until
34     from backend.storage import fetch, parse_uri
35     from backend.storage import get_storage
36
37     LOG = logging.getLogger("compute.cloud_run")
38
39     # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
40     # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
41     _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
42         "deployment.compute_target=local_gpu",
43         "indexing.detector_impl=native",
44     )
45
46
47     class CloudRunJobClient(ABC):
48         """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
49
50         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
51         overrides the job's container ``args`` for this execution and starts it."""
52
53         @abstractmethod
54         def run_job(self, job_name: str, argv: List[str], *, region: str,
55                     project: Optional[str] = None) -> str:
56             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
57             execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
58
59
60     class GoogleCloudRunJobClient(CloudRunJobClient):
61         """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
62         a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
63
64         def run_job(self, job_name: str, argv: List[str], *, region: str,
65                     project: Optional[str] = None) -> str:
66             try:
67                 from google.cloud import run_v2 # type: ignore[attr-defined]
68             except Exception as exc: # pragma: no cover - real SDK not present in CI
69                 raise RuntimeError(
70                     "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
71                     "control plane (it is intentionally NOT a CI/test dependency).")
72             from exc

```

```

73         client = run_v2.JobsClient() # pragma: no cover - exercised only on the real M7
run
74         name = client.job_path(project, region, job_name)
75         overrides = run_v2.RunJobRequest.Overrides(
76
container_overrides=[run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)]
77     )
78     op = client.run_job(request=run_v2.RunJobRequest(name=name,
overrides=overrides))
79     return getattr(op, "metadata", None) and getattr(op.metadata, "name", "") or
"execution"
80
81
82     class CloudRunCompute(ComputeBackend):
83         """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
84
85         def __init__(self, *, run_client: CloudRunJobClient, job_name: str, region: str,
86             project: Optional[str] = None, storage_config: Optional[dict] = None,
87             policy: ExecutionPolicy = DEFAULT_POLICY,
88             token: Optional[str] = None,
89             container_overrides: Optional[tuple[str, ...]] = None) -> None:
90             self._client = run_client
91             self._job_name = job_name
92             self._region = region
93             self._project = project
94             self._storage_config = storage_config or {}
95             self._policy = policy
96             # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
97             # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
98             self._token = token
99             self._container_overrides = (
100                 _DEFAULT_CONTAINER_OVERRIDES if container_overrides is None else
tuple(container_overrides)
101             )
102
103         def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
104             out_root = spec.out_uri.rstrip("/")
105             token = self._token or uuid.uuid4().hex
106             done_uri = f"{out_root}/_dispatch/{token}.done"
107             argv: List[str] = ["infer", "--video-uri", spec.video_uri, "--out-uri",
spec.out_uri,
108                 "--done-uri", done_uri]
109             if spec.segmenter:
110                 argv += ["--segmenter", spec.segmenter]
111             for kv in (*spec.config_overrides, *self._container_overrides):
112                 argv += ["--set", kv]
113
114             execution = self._client.run_job(self._job_name, argv, region=self._region,
115                 project=self._project)
116             LOG.info("cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
117                 self._job_name, execution, done_uri)
118
119             marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
120                 label="cloud-run-infer", logger=LOG)
121             video_id = str(marker.get("video_id", ""))
122             # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
123             # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
124             rc = int(marker.get("returncode", 1))
125             if not video_id:
126                 LOG.warning("cloud-run: job=%s completion marker present but
empty/unreadable (no ")

```

```

127         "video_id) ? treating as failure", self._job_name)
128         rc = rc or 1
129         outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
130         report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
131         LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id,
rc)
132         return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
133
134     # --- storage-backed completion signal (bucket-poll) ---
135     def _read_marker(self, done_uri: str) -> Optional[dict]:
136         """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
137         ref = parse_uri(done_uri)
138         if not get_storage(ref.backend, config=self._storage_config).exists(ref):
139             return None
140         return self._read_json(done_uri) or {} # empty dict still terminates the poll
(present == done)
141
142     def _read_json(self, uri: str) -> dict:
143         # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
144         # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
145         # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
146         try:
147             ref = parse_uri(uri)
148             dst = os.path.join(tempfile.mkdtemp(prefix="cr-read-"), ref.name or
"obj.json")
149             local = fetch(uri, dst, config=self._storage_config)
150             with open(local, "r", encoding="utf-8") as f:
151                 data: Any = json.load(f)
152             return data if isinstance(data, dict) else {}
153         except Exception:
154             LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
155             return {}
156

```

## 12. assistant (2026-06-22T15:00:48.718Z)

This is the critical finding source: ``outputs_uri = f"{out_root}/outputs/{video_id}/"`` uses the `**cloud-computed**` ``video_id``, while the API ingests under its `**local**` ``video_id``. Let me confirm the ``_finalize_reingest``, ``query_play_segments``, ``prune_segments``, and ``segment_watermarks`` semantics.

## 13. user (2026-06-22T15:00:49.708Z)

```

backend\main.py:283: def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int)
-> None:
backend\main.py:695: def query_play_segments(req: QueryRequest):
backend\infrastructure\database.py:424:     def segment_watermarks(self, video_id: str) ->
Tuple[int, int]:
backend\infrastructure\database.py:438:     def prune_segments(self, video_id: str, *, play_upto:
Optional[int] = None,
backend\infrastructure\database.py:459:     def add_play_segment(self, video_id: str, start_time:
float, end_time: float, server: Optional[str] = None, receiver: Optional[str] = None, metadata:
Optional[Dict[str, Any]] = None) -> Optional[int]:
backend\infrastructure\database.py:564:     def query_play_segments(self, video_id: str, filters:
Dict[str, Any]) -> List[Dict[str, Any]]:
tests\test_reporting_harvest.py:22:     def query_play_segments(self, video_id, filters):

```

## 14. assistant (2026-06-22T15:00:53.267Z)

There's a discrepancy: the `\_failed` call inside `\_maybe\_dispatch\_remote` uses `\_prune\_segments(video\_id, play\_after=play\_wm, cand\_after=cand\_wm)` but the database method signature shows `\_play\_upto` / `\_cand\_...`. Let me read both `\_prune\_segments` and `\_finalize\_reingest`.

#### 15. user (2026-06-22T15:00:54.256Z)

```
283         locale: Optional[str] = None
284
285     def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
286         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
287
288         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
289         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
290         and
291         keep the prior good results intact ? a failed/empty re-run never destroys prior
292         data.
293         """
294         if segments:
295             db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
296         else:
297             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
298
299     # --- API Endpoints ---
300     @app.get("/api/config")
301     def get_config():
302         # P0b: this endpoint is unauthenticated, so never serialize a credential-shaped
303         # field. Today
304         # AppConfig holds no secrets (the Gemini key is env-only; cf_access_aud is a public
305         # id), so this
306         # is byte-identical for the real fields ? but it fails-closed if a secret is ever
307         added (guarded
308         # by tests/test_redact.py + the /api/config redaction test).
309         if global_config is None:
310             return {}
311         return redact_secrets(global_config.model_dump(mode="json"))
312
313     @app.get("/api/health")
314     def health():
315         """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
316         (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
317         confirm the engine is reachable and which detector is wired. Never raises."""
318         sport = getattr(global_config, "sport", None)
319         indexing = getattr(global_config, "indexing", None)
320         return {
321             "status": "ok",
322             "sport": sport,
323             "default_segmenter": getattr(indexing, "default_segmenter", None),
324         }
```

#### 16. assistant (2026-06-22T15:00:57.272Z)

Now let me read `\_prune\_segments`, `\_segment\_watermarks`, `\_add\_play\_segment`, and `\_query\_play\_segments` in the database.

#### 17. user (2026-06-22T15:00:57.440Z)

```
424     def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
425         """Return (max play_segment id, max candidate_segment id) for a video (0 if
426         none).
```

```

427         Captured before a (re)segmentation so the run's new rows (id > watermark) can be
428         told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
429         """
430         with self._connect() as conn:
431             cur = conn.cursor()
432             p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
433                             (video_id,)).fetchone()[0]
434             c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
435                             (video_id,)).fetchone()[0]
436             return int(p), int(c)
437
438     def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
439                       play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
440                       cand_after: Optional[int] = None) -> None:
441         """Delete play/candidate segments by id range (events cascade from
play_segments).
442
443         Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
444         successful run (`*_upto` = the pre-run watermark), or drop the NEW partial
rows
445         and keep the OLD ones when the run failed/produced nothing (`*_after`).
446         """
447         with self._connect() as conn:
448             cur = conn.cursor()
449             if play_upto is not None:
450                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
451             if play_after is not None:
452                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
453             if cand_upto is not None:
454                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
455             if cand_after is not None:
456                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
457             conn.commit()
458
459     def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
460         try:
461             with self._connect() as conn:
462                 cursor = conn.cursor()
463                 cursor.execute(
464                     "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
465                     (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {})))
466                 )
467                 conn.commit()
468                 return cursor.lastrowid
469         except Exception as e:
470             logger.error(f"Failed to add play segment: {e}")
471             return None
472
473     def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
474         return self._execute_write(
475             "INSERT INTO events (segment_id, timestamp, type, player, details) VALUES
(?, ?, ?, ?, ?)",
476             (segment_id, timestamp, event_type, player, json.dumps(details or {})),

```

```

477         "Failed to add event",
478     )
479
480     def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
481                               end_time: float,
482                               chunk_index: Optional[int] = None, confidence:
483                               Optional[float] = None,
484                               stats: Optional[Dict[str, Any]] = None,
485                               detection_params: Optional[Dict[str, Any]] = None) ->
486                               Optional[int]:
487         """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
488         detector-specific dicts stored as JSON. Returns the new candidate id, or
489         None."""
490         try:
491             with self._connect() as conn:
492                 cursor = conn.cursor()
493                 cursor.execute(
494                     """INSERT INTO candidate_segments
495                     (video_id, detector, chunk_index, start_time, end_time, duration,
496                      confidence, stats, detection_params)
497                     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
498                     (video_id, detector, chunk_index, start_time, end_time, end_time -
499                      start_time,
500                      confidence, json.dumps(stats or {}), json.dumps(detection_params or
501                      {})))
502                 )
503                 conn.commit()
504                 return cursor.lastrowid
505         except Exception as e:
506             logger.error(f"Failed to add candidate segment: {e}")
507             return None
508
509     def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
510         """Record that a candidate detection was confirmed as a given play_segment."""
511         return self._execute_write(
512             "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id = ?",
513             (segment_id, candidate_id),
514             f"Failed to link candidate {candidate_id} to segment {segment_id}",
515         )
516
517     def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
518                                  only_unlabeled: bool = False) -> List[Dict[str, Any]]:
519         """Fetch candidate detections for the annotation / fine-tuning workflow.
520         `stats` and `detection_params` are deserialized from JSON."""
521         query = "SELECT * FROM candidate_segments WHERE video_id = ?"
522         params: List[Any] = [video_id]
523
524         if detector:
525             query += " AND detector = ?"
526             params.append(detector)
527         if only_unlabeled:
528             query += " AND human_label IS NULL"
529
530         query += " ORDER BY start_time"
531
532         candidates = []
533         with self._connect() as conn:
534             rows = conn.cursor().execute(query, params).fetchall()
535             for row in rows:
536                 d = dict(row)
537                 d["stats"] = json.loads(d["stats"] or "{}")
538                 d["detection_params"] = json.loads(d["detection_params"] or "{}")
539                 candidates.append(d)
540         return candidates

```

```

536     def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
537         with self._connect() as conn:
538             row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
539 (video_id,)).fetchone()
540             if row:
541                 d = dict(row)
542                 d["validation_results"] = json.loads(d["validation_results"] or "[]")
543                 return d
544             return None
545
546     def list_videos(self, limit: Optional[int] = None) -> List[Dict[str, Any]]:
547         """All indexed videos, most-recently-added first. Orders by rowid (a monotonic
548 insertion
549 counter, since id is a TEXT PK) rather than the 1-second-granularity
550 processed_at, which
551 would tie within a batch. Mirrors get_video's row shape (validation_results
552 parsed). Powers
553 GET /api/videos for the operator console (PACKAGING_PLAN.md P1)."""
554         sql = "SELECT * FROM videos ORDER BY rowid DESC"
555         params: tuple = ()
556         if limit is not None:
557             sql += " LIMIT ?"
558             params = (int(limit),)
559         with self._connect() as conn:
560             rows = conn.cursor().execute(sql, params).fetchall()
561             out: List[Dict[str, Any]] = []
562             for row in rows:
563                 d = dict(row)
564                 d["validation_results"] = json.loads(d["validation_results"] or "[]")
565                 out.append(d)
566             return out
567
568     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
569 List[Dict[str, Any]]:
570         """
571         Dynamically queries play segments based on filters:
572         - duration_min: float
573         - server: string
574         - receiver: string
575         - event_type: string (e.g. smash, foul)
576         """
577         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
578         params = [video_id]
579
580         if filters.get("duration_min"):
581             query += " AND r.duration >= ?"
582             params.append(filters["duration_min"])
583
584         if filters.get("server"):
585             query += " AND r.server = ?"
586             params.append(filters["server"])
587
588         if filters.get("receiver"):

```

## 18. user (2026-06-22T15:01:02.216Z)

```

583         if filters.get("receiver"):
584             query += " AND r.receiver = ?"
585             params.append(filters["receiver"])
586
587         if filters.get("event_type"):
588             # Check if there is an event of a specific type inside this segment
589             query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
590 e.type = ?)"
591             params.append(filters["event_type"])

```

```

591
592     segments_list = []
593     with self._connect() as conn:
594         cursor = conn.cursor()
595         rows = cursor.execute(query, params).fetchall()
596
597         for row in rows:
598             r_dict = dict(row)
599             r_id = r_dict["id"]
600             r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
601
602             # Fetch associated events
603             event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
604             r_dict["events"] = []
605             for er in event_rows:
606                 e_dict = dict(er)
607                 e_dict["details"] = json.loads(e_dict["details"] or "{}")
608                 r_dict["events"].append(e_dict)
609
610             segments_list.append(r_dict)
611
612     return segments_list
613
614     def clear_video_data(self, video_id: str):
615         with self._connect() as conn:
616             conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
617             conn.commit()
618
619     # --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
620     # Writers follow the repo convention (swallow + logger.error, return bool); the
621     # worker logs loudly on a failed transition so a job can't silently wedge.
622

```

## 19. assistant (2026-06-22T15:01:12.083Z)

Important: ``query_play_segments`` does NOT have an ``ORDER BY``. So the segments returned to ``_finalize_reingest`` and put in the response are in implicit rowid order (typically insertion order, which matches start time as inserted). Let me note this but verify ordering implications.

Now the critical question (1): does the API's local ``video_id`` ever differ from ``result.video_id``? The API ingests under the `**local**` ``video_id`` (``_ingest_remote_segments(video_id, result.outputs_uri, ...)``), but reads ``result.outputs_uri`` which is ``<out_root>/outputs/<cloud_video_id>/``. The cloud ``video_id`` is the MD5 the worker computed from the input bytes it fetched from the bucket.

Let me check how the API computes its local ``video_id`` versus what the cloud job gets. The API does ``local_video = storage.acquire_local_input(req.video_path, ...)`` then ``compute_video_id(local_video)``. The cloud job fetches ``spec.video_uri`` (= ``req.video_path``, a ``gs://`` URI) and hashes those bytes. Let me check ``acquire_local_input`` and ``compute_video_id`` to assess whether they can diverge.

## 20. user (2026-06-22T15:01:12.886Z)

```

backend\utils\hashing.py:23:def compute_video_id(filepath: str) -> str:
backend\storage\__init__.py:38:def resolve_input_ref(raw: str, storage_config: Any = None) ->
StorageRef:
backend\storage\__init__.py:59:def acquire_local_input(raw: str, storage_config: Any = None, *,

```

## 21. user (2026-06-22T15:01:15.945Z)

```

38     def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
39         """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input

```



```

40         config (`input_backend`/`input_root`). Raises ``ValueError`` for an unsupported
URI scheme ?
41         callers at the API/CLI boundary catch that and return a clean client error (never a
500).
42
43         Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
44         ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
45         sc = _storage_dict(storage_config)
46         uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
47                               input_root=sc.get("input_root", ""))
48         return parse_uri(uri)
49
50
51     def input_is_local(raw: str, storage_config: Any = None) -> bool:
52         """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
53         plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
54         that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
55         ``resolve_input_ref``)."""
56         return resolve_input_ref(raw, storage_config).backend == "local"
57
58
59     def acquire_local_input(raw: str, storage_config: Any = None, *,
60                             scratch_parent: "str | None" = None) -> str:
61         """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
62
63         Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
64         byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
65         non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
66         path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
67         regenerable from the source) and relies on OS temp reclamation ? the same pull
pattern the
68         worker's ``cmd_infer`` already ships."""
69         sc = _storage_dict(storage_config)
70         ref = resolve_input_ref(raw, sc)
71         if ref.backend == "local":
72             return ref.key # identity ? no temp, no copy, byte-identical to passing the
path directly
73         scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
74         return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
75
76
77     def _backend_class(name: str):
78         """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
79         if name == "local":
80             from backend.storage.local import LocalStorage
81             return LocalStorage
82         if name == "s3":
83             from backend.storage.s3 import S3Storage
84             return S3Storage
85         if name == "gcs":
86             from backend.storage.gcs import GCSStorage
87             return GCSStorage
88         if name == "gdrive":
89             from backend.storage.gdrive import GDriveStorage
90             return GDriveStorage
91         raise ValueError(f"unknown storage backend: {name!r} (expected

```

```

local[s3|gcs|gdrive]")
92
93
94     def get_storage(backend: Optional[str] = None, *,
95                     config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
96         """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
97         the implementation; its per-backend settings come from ``config[backend]`` merged
with
98         ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
99         cfg = config or {}
100         name = backend or cfg.get("backend", "local")
101         settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
102         settings.update(overrides)
103         return _backend_class(name)(**settings)
104
105
106     def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
107              config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
108         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
109         (no copy); for s3/gcs it downloads to ``dst``."""
110         ref = parse_uri(uri)
111         return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)
112
113
114     def put(local_path: "str | os.PathLike[str]", uri: str, *,
115            config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
str:
116         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
117         ref = parse_uri(uri)
118         get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
119         return uri
120
121
122     def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->
Iterator[str]:
123         """Yield the URI of every object under ``prefix_uri``."""
124         ref = parse_uri(prefix_uri)
125         for r in get_storage(ref.backend, config=config).list(ref):
126             yield r.uri
127

```

## 22. assistant (2026-06-22T15:01:30.800Z)

Now I understand the video\_id consistency picture. Both the API and the cloud worker hash the same bytes (the bucket object), so in the normal case `result.video\_id == local video\_id`. But the API \*does not check this\*. Let me confirm the `play\_wm` (destroy-after-confirm watermark) capture ordering relative to any cloud write, and confirm the `\_finalize\_reingest` failure path. Let me also check `get\_compute\_backend` to understand the default-OFF guarantee and how `play\_wm`/`cand\_wm` interact.

Let me check `play\_wm` capture timing. In `\_execute\_processing`, line 600 captures `play\_wm`, `cand\_wm` = db\_instance.segment\_watermarks(video\_id) BEFORE calling `\_maybe\_dispatch\_remote`. Good ? captured before any cloud write. And the cloud writes only happen in `\_ingest\_remote\_segments` (local DB writes), not to the bucket. So `play\_wm` is correctly captured before any DB row is added.

Now let me verify the failure-path correctness. In `\_maybe\_dispatch\_remote.\_failed`, it calls `prune\_segments(video\_id, play\_after=play\_wm, cand\_after=cand\_wm)`. This drops rows with `id > play\_wm`. Let me check the edge case: what about the `returncode != 0` path ? at that point, has `\_ingest\_remote\_segments` already run? No ? the `returncode != 0` check happens BEFORE

`\_ingest\_remote\_segments`. Good. So `\_failed` for rc!=0 drops nothing (no rows written yet), and preserves prior rows. Correct.

But there's a subtle ingest-then-fail gap. Let me trace: after `\_ingest\_remote\_segments` runs (writing new rows), the code computes `segments` and calls `\_finalize\_reingest`. There's no try/except around `\_ingest\_remote\_segments`. If it raises (e.g. bucket fetch fails, malformed CSV path), the exception propagates out of `\_maybe\_dispatch\_remote` up to `\_execute\_processing`'s segmenter `try` block. Let me check what catches it there.

Looking at lines 606-610: `\_maybe\_dispatch\_remote` is called, and if it raises, it's inside the `try:` at line 564 but there's no except for it ? only `finally`. Let me re-examine the structure around lines 611-619 ? the inner `try/except` is only around `segmenter.process\_video`. So an exception from `\_maybe\_dispatch\_remote` (line 606) propagates past the inner try (which it's before), up through the outer `try` (line 564) to the `finally` (line 646), then out to the caller.

Let me check whether the caller (job worker) restores segments on that raise. The concern: if `\_ingest\_remote\_segments` writes some rows then raises mid-way, those partial rows are NOT pruned. Let me check the caller of `\_execute\_processing`.

### 23. user (2026-06-22T15:01:31.484Z)

```
scripts\nightly_reelcount.py:350:          # backend/main.py:_execute_processing) ? play_segments
has an FK to videos.id, so a
tests\test_async_jobs.py:121:  """Item 5 positive: direct CLI-style path
(m._execute_processing) vs job-worker path
tests\test_async_jobs.py:135:    direct_out = m._execute_processing(req)
tests\test_async_jobs.py:161:    direct_f = m._execute_processing(req_f)
tests\test_async_jobs.py:175:    de = m._execute_processing(req_e)
tests\test_base.py:123:    Exercise the exact duck-typing patterns used in main.py (both the
_execute_processing
tests\test_base.py:169:    This allows the except blocks in main.py (_execute_processing:193-198
and the
backend\api\job_worker.py:9:every cross-function seam ? `db_instance`, `global_config`,
_execute_processing`,
backend\api\job_worker.py:46:    is the segmenter's `_execute_processing`, run UNDER the
destroy-after-confirm contract.
backend\api\job_worker.py:86:        result = _main._execute_processing(
tests\test_job_worker_module_seam.py:6:(`_arm_wake_timer`, `_get_executor`, `db_instance`,
_execute_processing`)
backend\main.py:439:def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
backend\main.py:762:        # for a bucket URI (the same seam the process path uses,
_execute_processing main.py:159).
docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:39:- Call sites in
`backend/main.py` (both the CLI path and `_execute_processing` ~213/746) perform duck-typing:
`isinstance(result, Failure)`, `hasattr(result, "value")` else result`.
docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:71:- `_execute_processing`
(the job worker body) is nearly identical to the large CLI block inside `main()` (~694-762).
docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:438: - The "parity" tests
were not real: both paths were client.post("/api/process") (same job worker), and asserts were
tautological (len==len on same expr). Must drive distinct: actual CLI-style
`backend.main._execute_processing(req)` (or main() CLI) **and** job-worker (`_run_claimed_job` /
drain) for same ProcessRequest; assert byte-identical DB rows/play_segments/reports (mod
timestamps). Cover Failure + empty branches.
docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:444:[Omitted long matching
line]
docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:452:[Omitted long matching
line]
docs\archives\HARDENING_LOOP_HANDOFF-COMPLETED-2026-06-14.md:229: - test_async_jobs.py: vid now
used in real query_play_segments(vid, {}) + result asserts; parity test rewritten to drive
m._execute_processing (direct/CLI) + client job path (inline ? _run_claimed_job) + explicit
create+mark+_run_claimed_job; branches covered (success/Failure/empty). Migration test
assertions restored additively.
docs\CODE_MAP.md:114:[Omitted long matching line]
```

## 24. user (2026-06-22T15:01:34.780Z)

```
70
71
72     def _run_claimed_job(job: Dict[str, Any]) -> None:
73         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
74         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
75
76         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
77         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
78         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
79         job self-scheduled and will be re-claimed when `next_poll_at` is due.
80         """
81         import backend.main as _main
82         db_instance = _main.db_instance
83         job_id = job["id"]
84         req = _main._job_to_request(job)
85         try:
86             result = _main._execute_processing(
87                 req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
88             if result.get("video_id"):
89                 db_instance.set_job_video_id(job_id, result["video_id"])
90                 db_instance.mark_job_done(job_id, result)
91                 _maybe_record_job_cost(job, result) # M8 cost ledger (default-OFF)
92         except _main.JobWaiting as w:
93             # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
94             logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
95             db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
96         except Exception as e:
97             logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
98             db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
99
100
101     def _maybe_record_job_cost(job: Dict[str, Any], result: Dict[str, Any]) -> None:
102         """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
103         ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
104         (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
pricebook, and
105         persist it.
106
107         ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet,
so even with
108         ``billing.enabled=True`` AND a real calibrated pricebook, every recorded cost is
**0** until a
109         FOLLOW-UP wires the worker to emit usage (gpu/wall/egress/gemini). M8 lands the
ledger + pricebook
110         + the recording seam; the usage-emission step is separate (the metering hooks on the
real run).
111         With the placeholder pricebook the cost is 0 regardless.
112
113         Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job``
is the
114         claimed row (carries ``owner_id``); ``result`` is the pipeline output."""
115         import backend.main as _main
116
117         billing_cfg = getattr(_main.global_config, "billing", None)
118         if not billing_cfg or not getattr(billing_cfg, "enabled", False):
119             return
120         try:
121             usage = (result or {}).get("usage") or {}
122             _main.db_instance.record_job_cost(
123                 job["id"], usage=usage, pricebook_version=billing_cfg.pricebook_version,
```

```

124         gpu_type=usage.get("gpu_type"), compute_backend=(result or
{ }).get("compute_backend"),
125         owner_id=job.get("owner_id"), margin=billing_cfg.margin,
126     )
127     logger.info("Recorded cost for job %s (pricebook=%s).", job["id"],
billing_cfg.pricebook_version)
128     except Exception as e: # never wedge a completed job on bookkeeping
129         logger.warning("Cost recording for job %s failed (non-fatal): %s",
job.get("id"), e)

```

## 25. assistant (2026-06-22T15:01:42.964Z)

Confirmed the key gap. When `\_execute\_processing` raises (line 96-98 in job\_worker), the worker just `mark\_job\_failed` ? it does NOT restore segments. The in-process segmenter path handles this itself with a try/except that calls `prune\_segments` (main.py line 614-619). But `\_maybe\_dispatch\_remote` has NO equivalent protection around `\_ingest\_remote\_segments`: if it partially writes rows then raises, those rows leak (they survive, polluting the video's segments alongside prior good rows). This is finding (2) ? the failure path does not prune correctly when the ingest itself raises.

Now let me verify the `play\_wm` capture relative to the `was\_reingest` re-ingest semantics, and check `result.video\_id` vs local `video\_id` divergence more concretely. The question: in what scenario do they differ? The cloud worker hashes the bucket object; the API hashes the locally-acquired copy of the **same** bucket object. Normally identical. But edge case: if `req.video\_path` is something `acquire\_local\_input` resolves differently than the cloud worker. They both pass `req.video\_path` through resolution. Let me verify the worker `infer` uses the same hashing. Let me check the worker's `cmd\_infer`.

## 26. user (2026-06-22T15:01:45.166Z)

```

<tool_use_error>Path does not exist: C:/Users/avidu/Projects/bhi-cloudrun/backend/worker.py.
Note: your current working directory is C:\Users\avidu\Projects\badminton-highlight-
indexer.</tool_use_error>

```

## 27. user (2026-06-22T15:01:47.592Z)

No files found

## 28. user (2026-06-22T15:01:51.721Z)

```

C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__init__.py
---
worker
---find---
backend/compute/cloud_run.py
backend/worker/__main__.py

```

## 29. user (2026-06-22T15:01:56.749Z)

```

C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:1: """Worker dispatcher ? pull
from storage ? run the pipeline ? push outputs back.
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:3:     python -m backend.worker
infer --video-uri <uri> --out-uri <prefix> \
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:42: from backend.utils.hashing
import compute_video_id
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:88: def
_write_report_json(video_id: str, segments: List[dict], status: str, path: str) -> None:
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:90:         json.dump({
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:91:             "video_id":
video_id,
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:120:     outputs are durably in
the bucket (the backend bucket-polls the done-marker). The dispatched
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:123:     A remote job that never
writes a completion marker (a failed/hung container) makes the bucket

```

```

C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:129:
video_uri=args.video_uri, out_uri=args.out_uri,
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:137:     logger.info("[worker]
remote compute done: video_id=%s rc=%d outputs=%s",
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:138:
result.video_id, result.returncode, result.outputs_uri)
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:139:     return result.returncode
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:142:def
_write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count: int,
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:144:     """M6: write a tiny JSON
completion marker to ``done_uri`` (via the storage layer) for a remote
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:147:         marker =
{"video_id": video_id, "returncode": returncode,
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:149:         local =
os.path.join(scratch, "_done.json")
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:151:
json.dump(marker, f)
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:152:         storage.put(local,
done_uri, config=storage_cfg)
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:153:
logger.info("[worker] wrote completion marker -> %s", done_uri)
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:155:
logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:183:     video_id =
compute_video_id(local_video)
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:189:     db.add_video(video_id,
local_video, "failed" if failures else "processed",
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:208:         result =
segmenter_cls(db, config).process_video(video_id, local_video, max_frames=args.max_frames)
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:222:         "infer
produced 0 segments for video_id=%s (segmenter=%s, detector_impl=%s). "
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:227:         video_id,
segmenter_actual, detector_impl,
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:232:         db=db,
video_id=video_id, video_path=local_video, config=config,
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:238:     # 5. SERIALIZE + 6. PUSH
(outputs/<video_id>/?). idempotent on video_id.
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:239:     out_local =
os.path.join(scratch, "outputs", video_id)
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:242:
_write_report_json(video_id, segments, status, os.path.join(out_local, "report.json"))
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:244:     out_prefix =
args.out_uri.rstrip("/")
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:247:         uri =
f"{out_prefix}/outputs/{video_id}/{fn}"
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:251:     print(f"[worker] infer
done: {len(segments)} segment(s); pushed {len(pushed)} artifact(s):")
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:255:     # M6: a remote
dispatcher passes --done-uri; write a completion marker so its bucket-poll
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:256:     # terminates (carries
video_id ? the dispatcher's output path + the worker rc). DEFAULT-OFF:
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:257:     # no --done-uri ? no
marker (unchanged local run). Written on SUCCESS only ? a failed container
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:259:     if getattr(args,
"done_uri", None):
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:260:
_write_done_marker(args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch)
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:417:         video_id =
compute_video_id(local_video)
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:425:         traj =
runner.run_predict(local_video, traj_dir, video_id=video_id)
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:443:         json.dump(specs, f,
indent=2)
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:455:     rc =
subprocess.run(cmd).returncode

```

```

C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:462:    out_prefix =
args.out_uri.rstrip("/")
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:468:    print(f"[worker] train
done: pushed {len(pushed)} artifact(s):")
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:481:    pi =
sub.add_parser("infer", help="run inference on one video URI and push outputs")
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:483:    pi.add_argument("--out-
uri", required=True, help="output prefix (outputs/<video_id>/? is appended)")
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:488:    pi.add_argument("--done-
uri", default=None,
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:489:
help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:499:    pt.add_argument("--out-
uri", required=True, help="output prefix (weights/<version>/ is appended)")

```

### 30. assistant (2026-06-22T15:02:02.928Z)

This is crucial: line 257-260 shows the done-marker is **\*\*written on SUCCESS only\*\***. Let me read the worker's infer body around 180-260 to understand the failure paths (returncode 2/3) and whether they write a marker.

### 31. user (2026-06-22T15:02:03.822Z)

```

160     _apply_overrides(config, args.set)
161     if not _run_startup_checks(config):
162         return 2
163
164     # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
165     # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
166     # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
167     # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
168     compute = get_compute_backend(config)
169     if compute is not None:
170         return _dispatch_remote(compute, args)
171
172     storage_cfg = config.storage.model_dump()
173
174     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
175     os.makedirs(scratch, exist_ok=True)
176     config.output.output_dir = scratch
177
178     # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
179     local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
180     print(f"[worker] pulled {args.video_uri} -> {local_video}")
181
182     # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
183     video_id = compute_video_id(local_video)
184
185     # 3. VALIDATE (same registry as the main CLI).
186     val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
187     failures = [r for r in val_results if not r.passed]
188     db = Database(os.path.join(scratch, config.output.db_name))
189     db.add_video(video_id, local_video, "failed" if failures else "processed",
[r.model_dump() for r in val_results])
190
191
192     segments: List[dict] = []
193     segmenter_actual = None

```

```

194     segmentation_ran = False
195     status = "failed"
196     try:
197         if failures:
198             print("[worker] validation FAILED: "
199                   + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
200             return 2
201         seg_name = args.segmenter or config.indexing.default_segmenter
202         segmenter_cls = segmenter_registry.get(seg_name)
203         if not segmenter_cls:
204             print(f"[worker] unknown segmenter: {seg_name!r} "
205                   f"(have {list(segmenter_registry.list_available())})")
206             return 2
207         segmenter_actual = seg_name
208         result = segmenter_cls(db, config).process_video(video_id, local_video,
209 max_frames=args.max_frames)
210         segmentation_ran = True
211         if isinstance(result, Failure):
212             print(f"[worker] segmenter FAILED: {result.message}")
213             return 3
214         segments = result.value if hasattr(result, "value") else result
215         status = "success"
216         # Loud, never-silent diagnostic: a 0-segment success is almost always a
217         # empty match ? the cold-start failure mode (the substrate detector yielded no
218         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
219         # for more).
220         if not segments:
221             detector_impl = (os.environ.get("RALLY_DETECTOR_IMPL")
222                             or getattr(config.indexing, "detector_impl", None) or
223 "auto")
224             logger.warning(
225                 "infer produced 0 segments for video_id=%s (segmenter=%s,
226 detector_impl=%s). "
227                 "The detector substrate yielded no usable trajectories/windows ? check
228 the detector "
229                 "logs above; common causes: native runner UNAVAILABLE
230 (weights/repo/WASB_PYTHON env), "
231                 "the WASB env produced no detections, or the input resolution differs
232 from the tuned "
233                 "proxy resolution. Re-run with -v for the native command + frame
234 counts.",
235                 video_id, segmenter_actual, detector_impl,
236             )
237         finally:
238             # Best-effort per-video observability report (never raises) ? parity with the
239             # main CLI.
240             emit_indexer_report(
241                 db=db, video_id=video_id, video_path=local_video, config=config,
242                 val_results=val_results, val_context=val_context,
243                 segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
244                 was_reingest=False, segmentation_ran=segmentation_ran,
245             )
246         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
247         out_local = os.path.join(scratch, "outputs", video_id)
248         os.makedirs(out_local, exist_ok=True)
249         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
250         _write_report_json(video_id, segments, status, os.path.join(out_local,
251 "report.json"))
252         out_prefix = args.out_uri.rstrip("/")
253         pushed = []
254         for fn in sorted(os.listdir(out_local)):

```



```

247         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
248         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
249         pushed.append(uri)
250
251     print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
252     for u in pushed:
253         print("    ", u)
254
255     # M6: a remote dispatcher passes --done-uri; write a completion marker so its
bucket-poll
256     # terminates (carries video_id ? the dispatcher's output path + the worker rc).
DEFAULT-OFF:
257     # no --done-uri ? no marker (unchanged local run). Written on SUCCESS only ? a
failed container
258     # job currently lets the dispatcher's bounded poll time out (a failure-marker is a
follow-up).
259     if getattr(args, "done_uri", None):
260         _write_done_marker(args.done_uri, video_id, 0, len(segments), status,
storage_cfg, scratch)
261     return 0
262
263
264     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a

```

### 32. assistant (2026-06-22T15:02:11.463Z)

This is a very important finding. The done-marker on the worker side:

1. Is written **only on success** (returncode 2/3 failures return early at lines 200/206/212, before line 259 marker write ? so no marker at all on validation/segmenter failure).
2. When it IS written, it always passes `returncode=0` hardcoded (line 260: `\_write\_done\_marker(args.done\_uri, video\_id, 0, ...)`).

So the `returncode != 0` check in `\_maybe\_dispatch\_remote` (main.py line 519) can essentially **never** see a non-zero returncode from the marker for a worker-level failure: validation/segmenter failures produce NO marker (? the dispatcher's `poll\_until` times out ? `CloudRunCompute` returns rc=4 via the empty/missing marker path... let me verify), and a success always writes rc=0.

Wait ? let me re-check. On worker validation failure (return 2), no marker is written. The dispatcher's `poll\_until` polls `\_read\_marker` which returns `None` until the marker exists. If it never exists, `poll\_until` eventually raises a timeout. Let me check what `poll\_until` does on timeout and whether `run\_infer` converts it to rc=4 as the docstring claims.

### 33. user (2026-06-22T15:02:13.778Z)

```

5:  for FAILURES (an exception) *and* HANGS (no return in time). A hung attempt's worker thread
is a daemon,
39:class PolicyTimeout(TimeoutError):
58:     return d
66:     return cls(**kw)
75:     return policy.backoff_base_sec * (2 ** attempt)
83:     Returns ``fn``'s value on success. Raises ``PolicyTimeout`` if the final attempt hung, or
re-raises the
111:         return box["val"]                # type: ignore[return-value]
122:     raise last_exc
125:def poll_until(check: Callable[[], Optional[T]], policy: ExecutionPolicy = DEFAULT_POLICY,
*,
129:     """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None`` value, or
raise
130:     ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
still in progress.
143:         return result
147:         raise PolicyTimeout(f"{label} not ready within

```

```
max_wait_sec={policy.max_wait_sec}s")
```

#### 34. assistant (2026-06-22T15:02:26.308Z)

So ``poll_until`` raises ``PolicyTimeout`` on timeout. But in ``cloud_run.py`` ``run_infer``, line 119 calls ``poll_until(...)`` WITHOUT catching ``PolicyTimeout``. So on a worker validation/segmenter failure (no marker ever written), ``run_infer`` raises ``PolicyTimeout`` ? it does NOT return `rc=4` as the base.py docstring claims. That exception propagates to ``_maybe_dispatch_remote``'s ``try/except Exception`` (main.py line 514), which catches it and calls ``_failed(...)``. So functionally it's handled (turns into a clean failure), but:

1. The base.py/cloud\_run.py docstrings claiming "rc=4 = remote dispatch timeout" are **\*\*inaccurate\*\*** ? ``run_infer`` never returns `rc=4`; it raises ``PolicyTimeout``. The `rc=4` path is dead/unreachable as written. Minor doc/contract inconsistency.
2. More importantly: a worker **\*\*validation failure\*\*** (bad video) on the cloud path turns into a multi-minute ``poll_until`` timeout instead of a fast, clear "video failed validation" ? because the worker writes no marker on `rc=2/3`. The user waits for the full ``max_wait_sec`` and gets a generic "cloud dispatch error" rather than the validation failure. That's a real edge-case UX/correctness gap, though arguably worker-side (pre-existing). Let me note it's in scope since the task asks about `returncode!=0` + exception paths.

Now let me verify the `rc=4` dead-path claim and ``get_compute_backend``.

#### 35. user (2026-06-22T15:02:27.192Z)

```
125     def poll_until(check: Callable[[], Optional[T]], policy: ExecutionPolicy =
126         DEFAULT_POLICY, *,
127             label: str = "op", logger: Optional[logging.Logger] = None,
128             sleep: Callable[[float], None] = _time.sleep,
129             monotonic: Callable[[], float] = _time.monotonic) -> T:
130         """Poll `check()` every `poll_every_n_sec` until it returns a non-`None`
131         value, or raise
132         `PolicyTimeout` after `max_wait_sec`. `check` returns `None` while the op is
133         still in progress.
134
135         Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
136         INFO/WARNING.
137
138         """
139         log = logger or LOG
140         start = monotonic()
141         polls = 0
142         log.info("%s: waiting (poll_every=%.0fs, max_wait=%.0fs)",
143             label, policy.poll_every_n_sec, policy.max_wait_sec)
144         while True:
145             result = check()
146             if result is not None:
147                 log.info("%s: ready after %d poll(s) / %.0fs", label, polls, monotonic() -
148                     start)
149                 return result
150             elapsed = monotonic() - start
151             if elapsed >= policy.max_wait_sec:
152                 log.warning("%s: still not ready after %.0fs (max_wait) ? giving up", label,
153                     elapsed)
154                 raise PolicyTimeout(f"{label} not ready within
155                 max_wait_sec={policy.max_wait_sec}s")
156             polls += 1
157             log.debug("%s: poll %d, %.0fs elapsed, not ready", label, polls, elapsed)
```

#### 36. assistant (2026-06-22T15:02:31.286Z)

Confirmed: ``poll_until`` raises ``PolicyTimeout``, and ``cloud_run.run_infer`` doesn't catch it, so `rc=4` is never produced ? the base.py contract docstring is inaccurate (minor). The exception is caught by ``_maybe_dispatch_remote``. Now let me check ``get_compute_backend`` and the

`get\_compute\_backend(global\_config)` vs `get\_compute\_backend(config)` consistency, plus the `notify` signature.

### 37. user (2026-06-22T15:02:32.313Z)

```
C:\Users\avidu\Projects\bhi-cloudrun\backend\compute\__init__.py:32:def get_compute_backend(
```

### 38. user (2026-06-22T15:02:35.333Z)

```
32     def get_compute_backend(
33         config: Any,
34         *,
35         run_client: Optional[Any] = None,
36         policy: ExecutionPolicy = DEFAULT_POLICY,
37         token: Optional[str] = None,
38     ) -> Optional[ComputeBackend]:
39         """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
40         in-process (the default-OFF path).
41
42         Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
43         Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
44         / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
45         pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
46         """
47         target = getattr(getattr(config, "deployment", None), "compute_target", "") or ""
48         if target != "cloud_run":
49             return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
50
51         # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
52         from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
53
54         job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
55         region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
56         project = os.environ.get("GOOGLE_CLOUD_PROJECT")
57         client = run_client if run_client is not None else GoogleCloudRunJobClient()
58         return CloudRunCompute(
59             run_client=client, job_name=job_name, region=region, project=project,
60             storage_config=_storage_dict(config), policy=policy, token=token,
61         )
62
```

### 39. assistant (2026-06-22T15:02:58.028Z)

Now let me check the `\_finalize\_reingest` empty-segments case for the cloud path. If the cloud job succeeds but produces **0 segments** (a valid-but-empty match, or the 0-segment misconfig warned about in the worker), then:

- `\_ingest\_remote\_segments` writes 0 rows (n=0).
- `segments = []` (no new rows with id > play\_wm).
- `\_finalize\_reingest(video\_id, [], play\_wm, cand\_wm)` ? since `segments` is empty, it calls `prune\_segments(play\_after=play\_wm, cand\_after=cand\_wm)` ? drops the NEW rows (none) and **keeps prior good rows**.

But the response returns `"status": "success"` with `"Successfully indexed 0 play segments"` and `play\_segments: []`. Meanwhile the DB still has the prior good rows. So the response is inconsistent with the DB state on a 0-segment cloud re-ingest: it claims success with 0 segments but the prior rows are preserved (a subsequent `/api/query` would return the OLD segments). This mirrors the in-process empty behavior somewhat, but the in-process success message says "indexed N play segments using" ? let me verify the in-process path returns the same potentially-

misleading state. Actually in-process: `segments = result.value` (the new run's segments). If empty, `\_finalize\_reingest` keeps old rows, and response `play\_segments: segments` = `[]`, message "indexed 0". Same behavior. So this is consistent with existing behavior ? not a new bug. Good, I'll not flag it.

Now, the most important divergence question (1). Let me carefully construct the scenario where `result.video\_id != local\_video\_id`:

The API computes `video\_id = compute\_video\_id(acquire\_local\_input(req.video\_path))` ? hashing the bucket bytes fetched locally. The cloud worker computes `video\_id = compute\_video\_id(storage.fetch(args.video\_uri))` ? hashing the bucket bytes fetched on the GPU box. **\*\*Same object, same MD5\*\*** ? normally identical.

When could they differ? If the bucket object is **\*\*mutated between\*\*** the API's fetch and the worker's fetch (e.g. the user overwrites `gs://.../match.mp4` concurrently), the two hashes differ. Then:

- The API ingests segments under the **\*\*API's\*\*** `video\_id` (local hash).
- But `result.outputs\_uri` = `- `\_ingest\_remote\_segments` reads from the **\*\*worker's\*\*** outputs\_uri (correct ? that's where the CSV is) but persists under the **\*\*API's\*\*** video\_id.

So the segments WOULD be ingested under the API's video\_id, and `query\_play\_segments(API\_video\_id)` would find them. The DB row `add\_video(API\_video\_id, ...)` was created at line 579. So in this mutation scenario, things are actually **\*\*self-consistent on the API side\*\*** ? the ingest reads the right CSV and writes under the API's id, which matches the video row. The only oddity: the report/marker's video\_id differs from the DB's. This is an extreme edge case (concurrent overwrite) and largely self-correcting.

But there's a more impactful angle: the code **\*\*never validates\*\*** `result.video\_id` against the local `video\_id`, and never logs a warning if they differ. Given the docstring in `ComputeJobSpec` explicitly says the worker "(re)computes from the SAME input bytes" ? relying on that invariant silently. If it's violated, the symptom is subtle (segments correct, but report.json under a different id, and any downstream that joins on the marker's video\_id mismatches). A defensive log/assert would help. This is a minor robustness finding.

Let me now double-check finding (3): is `segments` (built via `query\_play\_segments` filtered by `id > play\_wm`) correctly ordered? `query\_play\_segments` has no `ORDER BY`, so rows come back in SQLite's default order (rowid asc for a simple table scan, which equals insertion order = start-time order as the CSV is read). The CSV reader iterates rows in file order. So as long as `intervals.csv` is sorted by start, order is preserved. Let me verify `\_write\_intervals\_csv` writes sorted. And check finding (4): `segmentation\_ran=True` + `segmenter\_actual` for the report on the cloud path.

#### 40. user (2026-06-22T15:02:58.899Z)

```
60     def _apply_overrides(config: Any, sets: List[str]) -> None:
61         """Apply `a.b.c=value` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
62         for kv in sets or []:
63             key, sep, val = kv.partition("=")
64             if not sep:
65                 raise ValueError(f"--set expects k=v, got {kv!r}")
66             parts = key.split(".")
67             obj = config
68             for p in parts[:-1]:
69                 obj = getattr(obj, p)
70             setattr(obj, parts[-1], _coerce(val))
71
72
73     def _write_intervals_csv(segments: List[dict], path: str) -> None:
74         """Decimal-second `[start,end)` rallies + shot count / ending reason ? the join-
friendly
75         artifact (same schema as the rally-annotator labels)."""
76         with open(path, "w", newline="") as f:
77             w = csv.writer(f)
```

```

78         w.writerow(["start", "end", "shot_count", "ending_reason"])
79     for s in segments:
80         w.writerow([
81             f'{float(s.get("start_time", 0.0)):.3f}',
82             f'{float(s.get("end_time", 0.0)):.3f}',
83             s.get("shot_count", ""),
84             s.get("ending_reason", s.get("shot_count_confidence", "")),
85         ])
86
87

```

#### 41. assistant (2026-06-22T15:03:10.700Z)

`\_write\_intervals\_csv` writes segments in the order the segmenter produced them (typically start-ordered). The CSV is read in order by `\_ingest\_remote\_segments`, inserted in order, and queried back without ORDER BY (rowid asc = insertion order). So ordering is preserved in practice, but it's **fragile**: `query\_play\_segments` has no `ORDER BY start\_time` (unlike `query\_candidate\_segments` which does at line 524). The in-process path returns `result.value` directly (segmenter-ordered), but the cloud path round-trips through an unordered DB query. If SQLite ever returns rows non-rowid-order (e.g. after the prune deletes prior rows, or with a different query plan), the cloud reel could be out of order. This is a latent ordering finding worth noting (minor ? relies on implicit rowid ordering).

Now let me confirm the finding about `play\_wm` filter interacting with prior rows. On a **re-ingest** (was\_reingest=True), prior rows exist with id <= play\_wm. After `\_ingest\_remote\_segments` writes new rows (id > play\_wm), `segments = [s for s in query\_play\_segments(...) if id > play\_wm]` correctly selects ONLY new rows. Then `\_finalize\_reingest(segments non-empty)` ? `prune\_segments(play\_upto=play\_wm)` drops the old rows. Correct. Good.

Now let me verify finding (2) about `play\_wm` capture "before any cloud write". The task specifically asks "is play\_wm captured before any cloud write". `play\_wm` is captured at line 600, and `\_maybe\_dispatch\_remote` is called at line 606. Inside it, the only DB writes are in `\_ingest\_remote\_segments`. So yes, `play\_wm` is captured before any write. Correct.

Now let me verify the critical leak in the ingest-raises path by checking there's truly no try/except around `\_ingest\_remote\_segments` and that partial rows would leak. Let me re-read the dispatch function carefully.

#### 42. user (2026-06-22T15:03:11.234Z)

```

478     fall through to the in-process segmenter ? **DEFAULT-OFF**: ``get_compute_backend``
returns ``None``
479     for every non-``cloud_run`` target, so the in-process path stays byte-identical.""
480     from backend.compute import ComputeJobSpec, get_compute_backend
481
482     compute = get_compute_backend(global_config)
483     if compute is None:
484         return None # default-OFF ? run the in-process segmenter below (unchanged)
485
486     storage_cfg = getattr(global_config, "storage", None)
487
488     def _failed(msg: str) -> Dict[str, Any]:
489         # Shape-match the in-process segmenter-fail branch + restore prior good rows
(none written yet).
490         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
491         return {"video_id": video_id, "status": "failed", "message": msg,
492             "results": [r.model_dump() for r in val_results], "play_segments": []}
493
494     # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
495     try:
496         input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
497     except ValueError as e:
498         return _failed(f"cloud-serving: unresolvable input '{req.video_path}': {e}")

```

```

499         if input_ref.backend == "local":
500             return _failed("cloud-serving needs a cloud input (e.g. gs://?); a path local to
this machine "
501                             "can't be read by the Cloud Run job. Put the video in your bucket
first.")
502         out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
503         if not out_root:
504             return _failed("cloud-serving needs storage.output_root set to a bucket (e.g.
gs://your-bucket) "
505                             "? that's where the Cloud Run job writes the result.")
506
507         notify("dispatching (cloud_run)")
508         logger.info("[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
req.video_path, out_root)
509         spec = ComputeJobSpec(video_uri=req.video_path, out_uri=out_root,
segmenter=seg_name)
510         try:
511             result = compute.run_infer(spec) # dispatch the Cloud Run Job + bucket-poll to
completion
512         except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
513             logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
514             return _failed(f"cloud dispatch error: {e}")
515         if result.returncode != 0:
516             return _failed(f"cloud job failed (returncode {result.returncode}).")
517
518         notify("ingesting (cloud_run)")
519         storage_dict = storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else
{}
520         _ingest_remote_segments(video_id, result.outputs_uri, storage_dict)
521         # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
522         segments = [s for s in db_instance.query_play_segments(video_id, {})] if
int(s.get("id", 0)) > play_wm]
523         _finalize_reingest(video_id, segments, play_wm, cand_wm)
524         return {"video_id": video_id, "status": "success",
525                 "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
526                 "results": [r.model_dump() for r in val_results], "play_segments": segments}
527

```

### 43. assistant (2026-06-22T15:03:21.838Z)

Confirmed. Lines 518-520: `\_ingest\_remote\_segments` is called with NO try/except. If it raises AFTER writing some rows (e.g. the bucket fetch of intervals.csv succeeds, then a later `db\_instance.add\_play\_segment` fails transiently, or `result.outputs\_uri` is empty/malformed so fetch fails after partial writes), the partial new rows (id > play\_wm) survive ? they are NOT pruned. The exception propagates to `\_execute\_processing` line 606 (uncaught there, no surrounding try for the dispatch call), through `finally`, to `job\_worker.\_run\_claimed\_job` which just `mark\_job\_failed`. The prior good rows AND the partial new rows now coexist ? DB corruption of that video's segment set. This violates the destroy-after-confirm contract that the in-process path carefully preserves.

Note also an empty `result.outputs\_uri` edge: if the marker is read but had a video\_id and rc=0, but `report.json` missing ? actually `outputs\_uri` would be non-empty. But consider rc=0 with empty video\_id: cloud\_run sets `outputs\_uri=""` and forces rc to stay (line 124-128 sets rc = rc or 1, so rc becomes 1 ? caught by `returncode != 0`). So empty outputs\_uri with rc=0 can't reach ingest. Good. But what if `outputs\_uri` is non-empty yet the `intervals.csv` doesn't exist (worker pushed report.json but not intervals ? e.g. 0 segments still writes intervals though)? `\_ingest\_remote\_segments` would call `storage.fetch` on a missing object ? raises ? same unprotected leak path. So the missing-CSV case is real and hits the unprotected ingest.

Let me verify the test file to understand what's covered and whether any of these are intentionally deferred.

## 44. user (2026-06-22T15:03:22.834Z)

```

58         with open(os.path.join(outputs, "intervals.csv"), "w", newline="",
encoding="utf-8") as f:
59             w = csv.writer(f)
60             w.writerow(["start", "end", "shot_count", "ending_reason"])
61             for s, e, sc, er in self.rows:
62                 w.writerow([f"{s:.3f}", f"{e:.3f}", sc, er])
63             os.makedirs(os.path.dirname(done_uri), exist_ok=True)
64             with open(done_uri, "w", encoding="utf-8") as f:
65                 json.dump({"video_id": self.video_id, "returncode": self.returncode,
66                     "segment_count": len(self.rows), "status": "success"}, f)
67             return "exec-1"
68
69
70     @pytest.fixture
71     def cloud_env(tmp_path, monkeypatch):
72         """API wired for cloud_run with a LOCAL bucket (the local storage backend makes the
bucket read an
73         identity file read ? no GCS). A gs:// input is 'fetched' to a local fake file for
hash+validate."""
74         db = Database(str(tmp_path / "t.db"))
75         bucket = tmp_path / "bucket"
76         bucket.mkdir()
77         cfg = AppConfig.model_validate({
78             "deployment": {"compute_target": "cloud_run"},
79             "storage": {"backend": "local", "output_root": str(bucket)},
80         })
81         monkeypatch.setattr(m, "db_instance", db)
82         monkeypatch.setattr(m, "global_config", cfg)
83         monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
84         # A real gs:// input would be fetched to hash+validate; return a local fake so that
path is offline.
85         fake_local = tmp_path / "fetched.mp4"
86         fake_local.write_bytes(b"FAKEVIDEOBYTES")
87         monkeypatch.setattr(m.storage, "acquire_local_input", lambda path, scfg:
str(fake_local))
88         monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(),
{}))
89         return db, bucket, monkeypatch
90
91
92     def _inject_fake(monkeypatch, fake):
93         """Make backend.compute.get_compute_backend (re-imported inside
_ maybe_dispatch_remote) build a
94         real CloudRunCompute around the FAKE client + the fast no-wait policy + a fixed
token."""
95         real = compute_pkg.get_compute_backend
96         monkeypatch.setattr(compute_pkg, "get_compute_backend",
97             lambda config: real(config, run_client=fake, policy=_FAST,
token="tok"))
98
99
100     def _meta(row):
101         md = row.get("metadata")
102         return json.loads(md) if isinstance(md, str) else (md or {})
103
104
105     def test_cloud_dispatch_ingests_segments(cloud_env):
106         db, _bucket, monkeypatch = cloud_env
107         fake = _FakeCloudClient()
108         _inject_fake(monkeypatch, fake)
109
110         out = m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid"))
111
112         assert out["status"] == "success"

```



```

113     assert len(out["play_segments"]) == 2
114     rows = db.query_play_segments(out["video_id"], {})
115     assert len(rows) == 2
116     metas = [_meta(r) for r in rows]
117     assert all(md["source"] == "cloud_run" for md in metas)
118     assert {md.get("shot_count") for md in metas} == {5, 3}
119     assert {md.get("ending_reason") for md in metas} == {"winner", "unforced_error"}
120
121
122     def test_cloud_dispatch_builds_correct_spec(cloud_env):
123         _db, bucket, monkeypatch = cloud_env
124         fake = _FakeCloudClient()
125         _inject_fake(monkeypatch, fake)
126
127         m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
128 segmenter="wasb_hybrid"))
129
130         assert len(fake.calls) == 1 # exactly one Cloud Run Job execution
131         argv = fake.calls[0]
132         assert argv[argv.index("--video-uri") + 1] == "gs://b/clip.mp4"
133         assert argv[argv.index("--out-uri") + 1] == str(bucket)
134         assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
135         joined = " ".join(argv)
136         assert "deployment.compute_target=local_gpu" in joined # container runs INLINE
137         (never re-dispatch)
138         assert "indexing.detector_impl=native" in joined
139
140
141     def test_cloud_failure_marks_failed_no_rows(cloud_env):
142         db, _bucket, monkeypatch = cloud_env
143         # pre-seed a prior good row ? must be preserved on a cloud failure (destroy-AFTER-
144         confirm).
145         db.add_video("vidLocalPrior", "gs://b/clip.mp4", "processed", [])
146         fake = _FakeCloudClient(returncode=1)
147         _inject_fake(monkeypatch, fake)
148
149         out = m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
150 segmenter="wasb_hybrid"))
151         assert out["status"] == "failed"
152         assert "returncode 1" in out["message"]
153         assert out["play_segments"] == []
154         assert db.query_play_segments(out["video_id"], {}) == []
155
156
157     def test_cloud_requires_a_bucket_input(cloud_env, tmp_path):
158         _db, _bucket, monkeypatch = cloud_env
159         fake = _FakeCloudClient()
160         _inject_fake(monkeypatch, fake)
161         local_path = str(tmp_path / "local_only.mp4")
162
163         out = m._execute_processing(m.ProcessRequest(video_path=local_path,
164 segmenter="wasb_hybrid"))
165         assert out["status"] == "failed"
166         assert "cloud input" in out["message"]
167         assert fake.calls == [] # never dispatched a local-only path
168
169
170     def test_default_off_runs_in_process(tmp_path, monkeypatch):
171         """compute_target='' ? get_compute_backend returns None ? the in-process segmenter
172 runs
173         byte-identically (the gate is inert). No Cloud Run dispatch."""
174         db = Database(str(tmp_path / "t.db"))
175         cfg = AppConfig() # compute_target defaults to ""
176         monkeypatch.setattr(m, "db_instance", db)
177         monkeypatch.setattr(m, "global_config", cfg)

```

```

172 monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(),
{}}))
173 video = tmp_path / "v.mp4"
174 video.write_bytes(b"hashable-bytes")
175
176 class _InProcSeg:
177     def __init__(self, db_, cfg_):
178         self.db = db_
179
180     def process_video(self, vid, path, *a, **k):
181         sid = self.db.add_play_segment(vid, 0.0, 5.0)
182         return [{"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0,
"metadata": {}}]
183
184 monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
185 out = m._execute_processing(m.ProcessRequest(video_path=str(video),
segmenter="motion"))
186 assert out["status"] == "success"
187 assert len(out["play_segments"]) == 1
188 assert "cloud_run" not in out["message"]
189
190
191 def test_full_api_process_to_query_via_cloud(cloud_env):
192     """End-to-end through the public API: POST /api/process (gs:// input) ? the inline
worker dispatches
193     to (fake) Cloud Run ? job done ? POST /api/query returns the ingested segments."""
194     db, _bucket, monkeypatch = cloud_env
195     fake = _FakeCloudClient(video_id="vidCloud2")
196     _inject_fake(monkeypatch, fake)
197     client = TestClient(m.app)
198
199     r = client.post("/api/process", json={"video_path": "gs://b/clip.mp4", "segmenter":
"wasb_hybrid"})
200     assert r.status_code == 202
201     job_id = r.json()["job_id"]
202
203     job = client.get(f"/api/jobs/{job_id}").json()
204     assert job["status"] == "done"
205     video_id = job["video_id"]
206     assert video_id
207
208     q = client.post("/api/query", json={"video_id": video_id})
209     assert q.status_code == 200
210     segs = q.json()["play_segments"]
211     assert len(segs) == 2
212     assert all(_meta(s)["source"] == "cloud_run" for s in segs)
213

```

#### 45. assistant (2026-06-22T15:03:39.761Z)

The tests confirm the divergence concern: in the tests, the fake cloud client uses `video\_id="vidCloud"`/`"vidCloud2"` (the cloud-side id), while the API hashes `b"FAKEVIDEOBYTES"` ? a different local `video\_id`. The test asserts `out["video\_id"]` and queries under it. Let me verify what `out["video\_id"]` is ? it's the **local** id (line 524 `video\_id: video\_id`), and the ingest wrote under the local id. The test reads `intervals.csv` from `result.outputs\_uri` which uses `vidCloud`. So the test passes BECAUSE the ingest deliberately reads from the cloud outputs\_uri but writes under the local id ? proving the two ids genuinely differ in the test and the code handles it.

So the design is: **outputs\_uri** uses cloud video\_id (for reading the CSV), DB persistence uses local video\_id. This is intentional and works. The risk is only if the CLOUD job fails to find the input it expects ? but both fetch the same URI. The actual concern: there's no assertion the report.json's video\_id (cloud) ever needs to equal local. Given the test deliberately makes them differ and it works, this is **by design**, not a bug. I'll downgrade my video\_id concern to a minor robustness note (no validation/logging of the divergence; the marker/report.json land

under the cloud id, orphaned from the DB row under the local id).

Now let me confirm the report (finding 4): on the cloud success path, `segmentation\_ran=True` and `segmenter\_actual` is set. Line 597 sets `segmenter\_actual = seg\_name` BEFORE the dispatch (line 606), and line 608 sets `segmentation\_ran = True`. So the `finally` emits the report with both correct. Good. But on the cloud **failure** path (`remote` returns a failed dict, `remote` is not None ? True ? `segmentation\_ran=True`, return). Wait ? line 607-610: `if remote is not None: segmentation\_ran = True`. This sets `segmentation\_ran=True` even when the cloud job **failed** (`remote["status"]=="failed"`). Let me check what the in-process failure path does for `segmentation\_ran` to compare.

Looking at the in-process path: line 620 sets `segmentation\_ran = True` AFTER `process\_video` returns (so even a `Failure` result ? `segmentation\_ran=True`, since line 620 is before the `isinstance(result, Failure)` check at 622). So in-process, a segmenter that ran-and-failed still reports `segmentation\_ran=True`. But cloud failures include cases where the job was **never dispatched** (unresolvable input, local input, no out\_root, dispatch error) ? these set `segmentation\_ran=True` too via line 608, even though NO segmentation ran at all. That's a **report inaccuracy**: e.g. a "cloud-serving needs a cloud input" validation-style rejection reports `segmentation\_ran=True` in the observability report, when the in-process equivalent (a pre-segmentation rejection) would have `segmentation\_ran=False`. Let me confirm by checking what the report does with `segmentation\_ran`.

#### 46. user (2026-06-22T15:03:41.128Z)

```
C:\Users\avidu\Projects\bhi-cloudrun\backend\main.py:562: segmentation_ran = False
C:\Users\avidu\Projects\bhi-cloudrun\backend\main.py:608: segmentation_ran = True
C:\Users\avidu\Projects\bhi-cloudrun\backend\main.py:620: segmentation_ran = True
C:\Users\avidu\Projects\bhi-cloudrun\backend\main.py:653: was_reingest=was_reingest,
segmentation_ran=segmentation_ran,
C:\Users\avidu\Projects\bhi-cloudrun\backend\main.py:1127: segmentation_ran = False
C:\Users\avidu\Projects\bhi-cloudrun\backend\main.py:1152: segmentation_ran = True
C:\Users\avidu\Projects\bhi-cloudrun\backend\main.py:1169:
was_reingest=was_reingest, segmentation_ran=segmentation_ran,
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:194: segmentation_ran = False
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:209: segmentation_ran =
True
C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py:235:
was_reingest=False, segmentation_ran=segmentation_ran,
C:\Users\avidu\Projects\bhi-cloudrun\backend\reporting\__init__.py:38: segmentation_ran: bool
= True,
C:\Users\avidu\Projects\bhi-cloudrun\backend\reporting\__init__.py:69:
segmentation_ran=segmentation_ran,
C:\Users\avidu\Projects\bhi-cloudrun\backend\reporting\harvest.py:189: segmentation_ran: bool
= True,
C:\Users\avidu\Projects\bhi-cloudrun\backend\reporting\harvest.py:209: if
vm.get("fps_source") == "unknown" and segmentation_ran:
C:\Users\avidu\Projects\bhi-cloudrun\backend\reporting\harvest.py:218:
segmenter_actual, config_default, was_reingest, segmentation_ran)
C:\Users\avidu\Projects\bhi-cloudrun\backend\reporting\harvest.py:219: _ai_stage(rec, config,
segmenter_actual, play_segments, segmentation_ran, run_signals)
```

#### 47. user (2026-06-22T15:03:44.110Z)

```
200         video_id_kind="file_md5",
201         video_path=video_path,
202         spec=spec if spec is not None else load_spec(),
203         **recorder_kwargs,
204     )
205     vm = video_meta_from_context(val_context, video_path)
206     # If we never probed an fps but segmentation still ran, the segmenter must have
207     # used its internal fallback (30fps / 1280px) ? surface that as the footgun it is,
208     # AND show the assumed numeric value so the report reads "30 (fallback)" not "?"
209     # (fallback)".
210     if vm.get("fps_source") == "unknown" and segmentation_ran:
211         vm["fps_source"] = "fallback"
```

```

211         if vm.get("fps") is None:
212             vm["fps"] = 30.0 # matches the runners' fallback (_probe_fps_width -> 30.0
/ 1280)
213         rec.set_video_meta(**vm)
214
215         _validation_stage(rec, val_results)
216         config_default = (config.get("indexing") or {}).get("default_segmenter")
217         _segmentation_stage(rec, play_segments, candidates, segmenter_requested,
218                             segmenter_actual, config_default, was_reingest,
segmentation_ran)
219         _ai_stage(rec, config, segmenter_actual, play_segments, segmentation_ran,
run_signals)
220         _highlights(rec, play_segments, vm)
221
222         rec.finalize()
223         return rec
224

```

#### 48. assistant (2026-06-22T15:03:51.165Z)

So ``segmentation_ran=True`` on a never-dispatched cloud rejection would, for example, mislabel ``fps_source`` as "fallback" with an assumed 30fps (line 209-212) ? a misleading observability artifact when no segmentation actually occurred. This is a **minor report-fidelity bug** on the cloud pre-dispatch rejection paths (unresolvable/local input, missing `out_root`, dispatch error). The clean fix: only set ``segmentation_ran=True`` when the cloud job actually ran (i.e. ``remote["status"]=="success``), or distinguish dispatch-time rejections.

Now let me run the new test suite + a quick lint to confirm baseline health, then also check ``play_wm`/`cand_wm`` and the ``play_wm`` filter ``int(s.get("id", 0))`` robustness. Let me run the cloud dispatch tests.

#### 49. user (2026-06-22T15:03:55.845Z)

```

..... [100%]
6 passed in 2.58s

```

#### 50. assistant (2026-06-22T15:04:06.833Z)

Tests pass. Now let me verify one more edge: the ``_ingest_remote_segments`` uses ``storage.parse_uri(uri)`` and ``storage.fetch`` ? but with the **local** backend (as in tests), ``fetch`` is an identity passthrough and ``dst`` is ignored. With a real GCS backend, the ``outputs_uri`` is ``gs://...`` so ``parse_uri(uri).backend == "gcs"``. Fine.

Let me double-check the ``storage_dict`` construction at line 519: ``storage_cfg.model_dump()`` if `hasattr(storage_cfg, "model_dump") else {}`. ``storage_cfg = getattr(global_config, "storage", None)``. If ``storage`` is a Pydantic model it has ``model_dump``. Good. But note ``_ingest_remote_segments`` is passed ``storage_dict``, while ``resolve_input_ref`` (line 496) and ``out_root`` (line 502) use ``storage_cfg`` (the model). Consistent enough.

One more: the ``compute = get_compute_backend(global_config)`` ? but the worker path (``backend/worker/__main__.py:168``) also calls ``get_compute_backend(config)``. The dispatched container is forced to ``compute_target=local_gpu`` so it returns None there ? no recursion. Confirmed by line 135-136 of the test. Good.

Let me verify the in-process path's ``play_wm`` is also captured for the local-input rejection (the ``_failed`` with ``prune_segments(play_after=play_wm...)``). On the ``input_ref.backend == "local"`` rejection, ``prune_segments(play_after=play_wm, cand_after=cand_wm)`` runs ? harmless (no rows written) but it does execute a DELETE on ``id > play_wm``. Since nothing was written, deletes nothing. Fine. But note: this happens for EVERY pre-dispatch rejection including the dispatch-error case, which is correct defensive behavior.

Now let me confirm the lint baseline on the changed file to make sure I'm not flagging style the repo tolerates.

## 51. user (2026-06-22T15:04:08.059Z)

All checks passed!

## 52. assistant (2026-06-22T15:04:15.624Z)

Lint clean. I have enough to report. Let me compile the findings. Let me re-verify the worker's done-marker hardcoded `rc=0` claim once more and that there's no failure marker, confirming the `rc!=0` branch is essentially dead for worker-level failures.

The worker writes the marker only at line 259-260 (after ``return 0`` would-be path ? actually it's right before ``return 0`` at 261), with hardcoded ``0``. Validation failure returns 2 at line 200 (inside the try, before the marker), segmenter failure returns 3 at line 212 ? both before line 259. So **no marker** on rc 2/3 ? dispatcher ``poll_until`` times out ? ``PolicyTimeout`` raised ? caught as "cloud dispatch error". And the only marker ever written carries `rc=0`. Therefore ``result.returncode != 0`` (main.py:515) is effectively **unreachable** for genuine job failures. This is the most significant correctness gap alongside the ingest-leak.

Let me record findings.

## 53. user (2026-06-22T15:04:51.086Z)

Structured output provided successfully

## 54. assistant (2026-06-22T15:05:03.738Z)

I reviewed the cloud-dispatch wiring on branch ``feat/serving-api-cloud-dispatch`` in ``C:\Users\avidu\Projects\bhi-cloudrun``. Five findings reported via StructuredOutput.

Summary of what I verified and found:

**Two major findings:**

- Partial-ingest row leak** (``backend/main.py:518-523``) ? ``_ingest_remote_segments`` has no try/except, so if it raises after writing some rows (notably a missing/partial ``intervals.csv`` in the bucket), the partial new rows survive alongside prior good rows, breaking the destroy-after-confirm contract the in-process path upholds at ``main.py:614-619``. The raise propagates uncaught to ``job_worker._run_claimed_job``, which only marks the job failed.
- ``returncode != 0`` gate is effectively dead** (``backend/main.py:515-516``) ? the worker (``backend/worker/_main_.py:259-260``) writes the done-marker only on success and hardcodes ``returncode=0``; validation/segmenter failures (rc 2/3) return before any marker is written. So ``run_infer`` hits ``PolicyTimeout`` (never the documented `rc=4`) after the full ``max_wait_sec``, surfacing as a generic "cloud dispatch error" instead of a fast, accurate failure.

**Three minor findings:** ``segmentation_ran=True`` set on cloud pre-dispatch rejections (corrupts the fps report); the cloud-vs-local ``video_id`` divergence is silently tolerated (the design reads cloud ``outputs_uri`` but writes under the local id ? works, but unguarded); and the cloud segments are returned via ``query_play_segments`` which has no ``ORDER BY``, so reel ordering relies on an implicit rowid invariant.

What I verified as correct: ``play_wm`/`cand_wm`` are captured (line 600) before any DB write; the ``_failed`` failure path prunes with ``play_after`/`cand_after`` and preserves prior rows; the ``id > play_wm`` filter correctly isolates new rows; the `rc!=0` and dispatch-exception paths preserve prior rows; default-OFF is intact (``get_compute_backend`` returns None for non-``cloud_run``); and the 6 tests in ``tests/test_api_cloud_dispatch.py`` pass with clean ruff.